

Basi di Dati

*I seguenti appunti sono a cura di **Catalin Ceban** e si basano sul corso di "Basi di Dati" erogato nell'A.A. 2026 dal professore Giuseppe Perelli per il C.d.L Informatica (33503)*

27 settembre 2026

Indice

1	La memorizzazione dei dati nel passato	2
1.1	Sistema informativo e sistema informatico	2
1.2	La gestione dei dati prima dei DBMS	3
2	Basi di dati e DBMS	4
2.1	Architettura ANSI/SPARC	7
2.1.1	Schema esterno	7
2.1.2	Schema logico	7
2.1.3	Schema fisico	7
2.2	Le transazioni	8
3	Progettazione di basi di dati	11
3.1	Modelli informatici	12
3.2	Entità, collezioni e attributi	13
3.3	Le associazioni	14
3.4	Le gerarchie di classi	16
3.4.1	Vincoli su insiemi di sottoclassi	18

Capitolo 1

La memorizzazione dei dati nel passato

Si ritiene opportuno, come introduzione al corso, fornire vari esempi e una definizione abbastanza comune di "base di dati". Interagiamo con basi di dati costantemente durante la nostra quotidianità: accedendo all'applicazione della nostra banca, utilizzando i social network, effettuando un acquisto su piattaforme di E-Commerce, ...

Questi sono solo pochi degli innumerevoli esempi di basi di dati con le quali ci troviamo a contatto ogni giorno. Possiamo dunque concludere che una definizione molto poco formale e spiccola di "base di dati" sia letteralmente *un grande e organizzato contenitore di dati di alcun tipo*.

1.1 Sistema informativo e sistema informatico

Nelle situazioni sopra elencate, ma anche nella vita di tutti i giorni, è strettamente necessario tenere traccia delle informazioni riguardanti persone o cose. Pensiamo ad esempio ad un'azienda: all'interno di questa sarà necessario tenere traccia di tutti i dettagli anagrafici dei dipendenti, ma anche dei contratti, degli stipendi, ecc...

Tutti questi dati venivano e vengono ancora oggi gestiti dal cosiddetto **sistema informativo**.

Definizione 1.1.1: Sistema informativo

Il *sistema informativo* è l'insieme di quelle risorse, umane o materiali, che un'organizzazione impiega per la gestione e la memorizzazione delle informazioni.

Un banale esempio di sistema informativo potrebbe essere un cassetto contenente delle cartelle, ognuna relativa ad ogni dipendente (risorsa materiale) e la segretaria che è adibita all'accesso e al controllo di questi dati.

Come ben sappiamo però, l'epoca degli archivi fisici è da un po' passata: con l'avvento della tecnologia, e in particolare con la sua massiccia distribuzione, le organizzazioni si sono sempre di più affidate a strumenti di memorizzazione automatizzati e a sistemi per la gestione semi-automatica dei dati (gli elaboratori,

per l'appunto). Nasce così un altro importante concetto, quello del **sistema informatico**.

Definizione 1.1.2: Sistema informatico

Il *sistema informatico* è l'insieme delle tecnologie utilizzate dall'organizzazione per la memorizzazione dei dati. Parliamo anche di **sistema informativo automatizzato** quando includiamo nella precedente definizione di sistema informativo anche le componenti elettroniche.

1.2 La gestione dei dati prima dei DBMS

Prima della nascita e della diffusione dei moderni sistemi di gestione di basi di dati (DBMS), i processi di organizzazione e memorizzazione delle informazioni non venivano ancora effettuati nel modo più efficiente possibile. Infatti, ogni organizzazione aveva il proprio **file privato** in cui memorizzare i dati. Utilizzando dunque un singolo file per ogni informazione, la memorizzazione dei dati avveniva in modo *sequenziale*. Dunque, per reperire un dato, era anche necessario scorrere ogni record del file finché questo non fosse stato trovato. Gli svantaggi di questa metodologia sono evidenti ad occhio nudo:

- **Ridondanza:** se lo stesso dato doveva essere condiviso tra più applicazioni, questo doveva essere duplicato (letteralmente un copia e incolla).
- **Inconsistenza:** a causa della ridondanza, aggiornando un'istanza di un record ci si poteva dimenticare di aggiornare anche le altre.
- **Dipendenza dall'applicazione:** ogni applicazione era fortemente legata alla struttura fisica delle cartelle e dei dati in suo possesso.

Capitolo 2

Basi di dati e DBMS

Avendo fornito una panoramica riguardo ai sistemi e alle strategie di memorizzazione utilizzate nel passato (si parla prima degli anni '70), è giusto concentrarsi sullo standard adottato da tali anni fino ad oggi. Iniziamo fornendo una definizione più formale di **base di dati**.

Definizione 2.0.1: Base di dati

Una *base di dati* è una raccolta di dati permanenti, gestita da un elaboratore elettronico.

Più nel pratico, una base di dati rappresenta un insieme di file mutualmente connessi, gestiti efficientemente in modo da rendere l'accesso ai dati il più rapido possibile. Le basi di dati permettono la memorizzazione di due tipi di dati:

- **Metadati:** delle informazioni speciali che descrivono lo schema della base di dati (la sua struttura) e tutti i vincoli e le relazioni che intercorrono tra i vari file.
- **Dati:** le effettive informazioni che, contestualizzate, vengono inserite e memorizzate nei file. Nota importante sulla parola *contestualizzate*: un numero o una stringa da soli non significano niente. Solo una volta che identifichiamo il numero come ad esempio la matricola di uno studente e la stringa come il suo nome questi dati assumono di significato e possono essere memorizzati con un criterio all'interno dei file.

Assieme al concetto di base di dati presentiamo un altro concetto fondamentale che viaggia di pari passo con il precedente:

Definizione 2.0.2: DataBase Management System (DBMS)

Il *DBMS* è uno strumento software che si occupa principalmente di gestire grandi quantità di dati. Un buon DBMS deve inoltre fornire linguaggi e strumenti per permettere le seguenti operazioni:

- Definizione delle relazioni tra i dati.
- Inserimento/modifica/cancellazione dei dati nei file.

- Creazione dei vincoli di integrità che devono essere stabiliti tra i record in relazione.
- Memorizzazione fisica delle informazioni tramite apposite strutture dati.

Inoltre, un DBMS deve garantire due proprietà fondamentali:

- **Indipendenza fisica:** modificando l'organizzazione della base di dati, i programmi applicativi non devono risentire di modifiche o malfunzionamenti.
- **Indipendenza logica:** i programmi applicativi non devono essere modificati a seguito di una modifica nello schema logico.

Tale principio di indipendenza è fondamentale per eliminare quella dipendenza stretta tra supporto fisico e software che tanto aveva causato problemi nell'epoca pre-DBMS.

Nel mondo dell'informatica, vigono due modalità di uso principali dei DBMS:

On-Line Transaction Processing (OLTP): si tratta della metodologia classica di utilizzo dei DBMS, che prevede lo sfruttamento completo dei vincoli di integrità messi a disposizione da questo. Le organizzazioni che fanno uso di questo paradigma si concentrano maggiormente su *quali dati e come sono memorizzati*, e le operazioni svolte (*transazioni*) sono spesso molto semplici e lavorano su relativamente pochi dati.

On-Line Analytical Processing (OLAP): secondo questo paradigma è importante che i dati siano molti e vengano memorizzati in apposite strutture. Infatti, le organizzazioni che fanno uso di questa modalità tendono a sfruttare le *data warehouses* (letteralmente dei "magazzini per i dati") in quanto sono maggiormente utilizzati nell'ambito della ricerca e dell'analisi scientifica, dove non importa *come* sono organizzati i dati bensì *quali* dati sono memorizzati.

Tuttavia la vera differenza nell'uso dei DBMS venne fatta dal matematico Edgar F. Codd intorno agli anni '70, quando propose di creare un modello di DBMS che si ispirasse al concetto matematico di **relazione** (che ricordiamo essere un sottoinsieme del prodotto cartesiano tra due insiemi). Nasce così il **modello relazionale**.

Definizione 2.0.3: DBMS relazionale

Un DBMS che implementa il modello relazionale organizza i propri dati all'interno di n -uple ordinate, dette **record**, in cui ogni elemento del record rappresenta una particolare informazione.

Nella definizione di una base di dati relazionale bisogna anche definire uno **schema** secondo il quale organizzare questi dati. Tale schema non è altro che la *struttura logica* della tabella, cioè una n -upla di quelli che sono gli attributi (o più informalmente, le colonne) della tabella. Infine, possiamo dire che una tabella contiene tutti record di tipo omogeneo, cioè tutti record che hanno la stessa struttura (o, appunto, lo stesso schema).

All'interno del modello relazionale, due record non vengono collegati tramite puntatori come ad esempio nel modello gerarchico o reticolare (vedi dopo) bensì viene stabilito un collegamento tra due attributi dei due record che devono necessariamente essere dello stesso tipo e contenere lo stesso valore. Intuitivamente si può pensare che tale procedimento causi una forte ridondanza dei dati, ma tale osservazione è parzialmente errata in quanto:

- Un buon DBMS deve mettere a disposizione gli strumenti per la creazione di tali collegamenti e controllare che questi non vengano violati (**vincoli di integrità**).
- Ogni record è memorizzato **una e una sola volta** sul disco fisico. Ciò vuol dire che se un record studente è associato ad una classe, non verrà duplicato alcun record per stabilire il collegamento, ma semplicemente verranno collegati i due record già esistenti. Tale ridondanza deve però essere gestita, come detto anticipatamente, dal DBMS.

Matricola	Cognome	E-Mail
2671072	Rossi	rossi@sapienza.it
2623072	Bianchi	bianchi@sapienza.it
2623272	Verdi	verdi@sapienza.it

Sopra è presentato un esempio di come vengono organizzati i dati all'interno di una tabella nel modello relazionale. Come possiamo vedere ogni record ha la stessa struttura: matricola, cognome, e-mail. Questa struttura inoltre è proprio lo *schema della tabella* (inclusi vari vincoli di integrità stabiliti).

Tuttavia il modello relazionale non fu e ancora oggi non è l'unico modello per la rappresentazione dei dati.

Modello reticolare: i dati vengono presentati come una collezione di record di tipo omogeneo (simile al relazionale). Tuttavia, le effettive *relazioni* tra i

vari record venivano fatte tramite dei *puntatori* da un record ad un altro.

Modello gerarchico: si tratta di una sottoclasse del modello reticolare, e prevede l'organizzazione dei dati in una struttura simile a quella di un albero informatico, dove ogni nodo (record) ha solo un genitore.

2.1 Architettura ANSI/SPARC

Nello studio dei sistemi di gestione di basi di dati, risulta particolarmente utile ed efficace distinguere tre differenti livelli di descrizione dei dati, noti comunemente come schemi. Tale suddivisione permette di garantire l'indipendenza dei dati e di gestire la complessità strutturale del sistema. Vediamo più nel dettaglio ciascun livello componente questa architettura.

2.1.1 Schema esterno

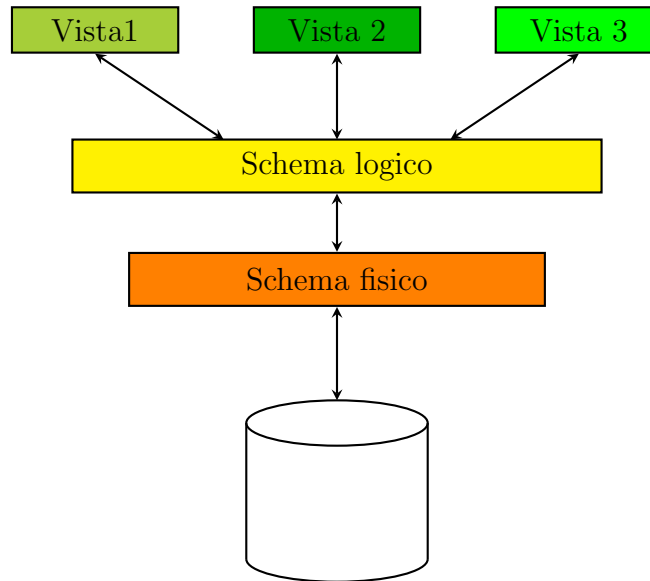
Si tratta della porzione più superficiale dell'architettura, quella con cui saranno a contatto gli utenti che non dovranno avere nulla a che fare con la gestione dei dati, ma solo con il loro utilizzo (ad esempio, la segretaria di un ufficio). Tale schema predispone anche che ogni utente sia in grado di visualizzare solo *una porzione* delle informazioni contenute nella base di dati, in base ai suoi permessi.

2.1.2 Schema logico

Questo livello descrive la struttura degli insiemi di dati e delle relazioni tra loro, secondo il modello adottato dal DBMS. Nota che questo livello si occupa di definire lo schema dei dati, ma non ha alcuna influenza su come questi vengono memorizzati fisicamente in memoria. Un esempio di schema logico può essere il seguente, che definisce la struttura di una tabella all'interno della base di dati di un'università; `Studenti(matricola char(8), cognome char(20), annoDiNascita int)`

2.1.3 Schema fisico

L'ultimo e più profondo livello si occupa di gestire l'effettiva memorizzazione dei dati all'interno dei supporti di memorizzazione, fornendo le strutture dati necessarie per facilitarne l'accesso (in particolare i B-Alberi). Ad esempio, l'amministratore della base di dati potrebbe decidere che i dati vadano *memorizzati* in maniera sequenziale (i record vengono messi uno dopo l'altro) e l'accesso a questi venga fatto tramite un indice sull'identificativo univoco del record.



2.2 Le transazioni

Un buon DBMS deve inoltre gestire anche una serie di operazioni sensibili di inserimento, modifica e cancellazione di dati, in modo che queste vengano effettuate senza contrattempi o errori.

Definizione 2.2.1: Transazione

Una **transazione** è una sequenza di operazioni di lettura e scrittura in memoria che deve rispettare le seguenti proprietà **ACID**:

- **Atomicity:** deve essere una sequenza **indivisibile** di operazioni. In caso di verifica di un qualsiasi errore nel mezzo di una transazione porta questa ad essere marcata come *non valida*, in quanto le operazioni devono essere eseguite **tutte o nessuna**.
- **Consistency:** bisogna lasciare i dati modificati in uno stato corretto che non viola i vincoli imposti precedentemente dal DBMS.
- **Isolation:** ogni transazione deve essere **indipendente**. Dunque durante esecuzione concorrente di più transazioni, gli effetti di una non devono dipendere dalle altre.
- **Durability:** gli effetti di una transazione devono essere **permanenti** per quanto riguarda i dati modificati (ovvero le modifiche devono necessariamente essere applicate anche su disco per mantenere la sincronizzazione dei dati).

Vediamo di seguito un esempio di due transazioni che vengono eseguite bene o

male nello stesso momento:

- **Transazione 1:** "Accredita 1000 € sul conto corrente *c1*"
- **Transazione 2:** "Accredita 500 € sul conto corrente *c1*"

Supponiamo che il valore iniziale del saldo del conto *c1* sia pari a 2000 €. Intuitivamente, al termine di entrambe le transazioni, ci aspetterions un valore finale di 3500 €. Vediamo invece cosa può accadere se le operazioni vengono interlacciate nel tempo senza opportune sue protezioni.

Osservazione 2.2.1

Una volta letto un valore, ogni transazione lo modifica nel **proprio** spazio di memoria di lavoro; altrimenti si rischiano modifiche irreversibili ai dati nello schema.

Analizziamo la seguente tabella di esecuzione temporale delle due transazioni:

Transazione 1	Tempo	Transazione 2
cerca <i>c1</i>	<i>t1</i>	
	<i>t2</i>	cerca <i>c1</i> ← <i>qui non prende il valore aggiornato +1000</i>
modifica <i>saldo</i> in <i>saldo</i> +1000	<i>t3</i>	
	<i>t4</i>	modifica <i>saldo</i> in <i>saldo</i> +500

Come si può vedere dalla tabella, l'esecuzione concorrente di due transazioni, in questo caso, risulta in un esito errato, in quanto ogni transazione effettua modifiche sulla **sua** porzione di dati, e non su quella effettivamente memorizzata, il che causa problemi di sincronizzazione.

Definizione 2.2.2: Serializzabilità

Nel caso di esecuzioni concorrenti di più transazioni, l'effetto complessivo deve essere equivalente a quello di un'esecuzione seriale, ovvero con le transazioni eseguite *una dopo l'altra*, senza alcun interfogliamento delle loro operazioni elementari.

Chiaramente la serializzabilità non è sempre semplice da implementare, e si fa infatti uso di protocolli progettati per garantire a priori la correttezza di qualunque schedule di transazioni che li adottano. Queste tecniche e protocolli non saranno tuttavia argomento di questo corso, per cui si lascia al lettore la voglia di approfondire.

Continuando l'analisi delle funzionalità offerte da un DBMS, è fondamentale esaminare gli strumenti messi a disposizione per la gestione e lo sviluppo. Tali funzionalità non si rivolgono unicamente agli utenti finali che eseguono interrogazioni sui dati, ma coprono ruoli cruciali come l'amministratore della base di dati e lo sviluppatore di applicazioni.

Concludendo questo capitolo, se volessimo elencare i vantaggi e gli svantaggi nell'utilizzo di un DBMS *ben progettato* per la memorizzazione dei dati, otterremmo un simile resoconto.

Osservazione 2.2.2: Vantaggi nell'uso di un DBMS

1. Indipendenza fisica e logica tra dati e applicazioni che li usano.
2. Memorizzazione e recupero efficiente dei dati.
3. Vincoli di integrità sui dati.

Il principale **svantaggio** nell'utilizzo di un DBMS risiede nella stretta necessità di definire uno schema di dati prima di poterli memorizzare. Tale punto diventa uno svantaggio nel momento in cui si vogliono memorizzare, ad esempio, piccole quantità di dati (ad esempio se vogliamo memorizzare il contenuto di un libro), oppure non ci interessa *come* questi dati vengano memorizzati.

Capitolo 3

Progettazione di basi di dati

Progettare una base di dati significa progettare sia la **struttura dei dati** sia le **applicazioni** che vi operano sopra. Tra queste due componenti, la **progettazione dei dati** rappresenta l'attività più importante e delicata. Tale progettazione non è un'attività fine a sé stessa, bensì rientra nel **ciclo di vita di un sistema informativo**.

Definizione 3.0.1: Ciclo di vita di un sistema informativo

Il *ciclo di vita di un sistema informativo* è quella sequenza di attività svolte da analisti, progettisti e utenti nello sviluppo dei sistemi informativi. Viene detto *ciclo* in quanto ogni fase non è una tantum, bensì può essere ripetuta più volte apportando le opportune modifiche al fine di raggiungere il risultato finale sperato.

Tale ciclo si articola nelle seguenti fasi:

1. **Studio di fattibilità:** definizione delle proprietà, stime dei costi, valutazione dei tempi richiesti dal progetto.
2. **Raccolta e analisi dei requisiti**
3. **Progettazione e implementazione:** modellazione concettuale e logica dei dati che si vogliono memorizzare e implementazione tramite un apposito linguaggio DBMS.
4. **Validazione e collaudo:** effettuazione dei test per verificare che il sistema risponda correttamente alle esigenze.

La modellazione di una base di dati si colloca a *metà* tra la raccolta e l'analisi dei requisiti e la progettazione effettiva.

3.1 Modelli informatici

Prima di modellare la base di dati, bisogna decidere a quale modello informatico/matematico rifarsi per la memorizzazione di questi. Un modello deve presentare le seguenti caratteristiche:

- Deve essere **la rappresentazione di un certo fatto concreto**.
- Tale rappresentazione deve essere descritta mediante un **linguaggio formale** (dunque che segue delle regole ben definite).
- Individuazione delle strutture da utilizzare per organizzare i dati di interesse e le relazioni tra questi.
Ad esempio, il modello relazionale di cui abbiamo discusso prima utilizza il concetto matematico di *relazione* per organizzare i dati.

Possiamo definire due tipologie di modelli fondamentali nella progettazione di una base di dati:

Definizione 3.1.1: Modello logico e modello concettuale

Il **modello logico** di una base di dati rappresenta la struttura dei dati e come questi vengono memorizzati. Si potrebbe dire che corrisponde allo schema logico visto precedentemente nel modello ANSI/SPARC, in quanto si colloca indipendente dalla memorizzazione fisica dei dati, ma è strettamente legato dal tipo di modello informatico che abbiamo scelto di adottare per la memorizzazione di questi.

Il **modello concettuale** è tutto il contrario: deve essere completamente indipendente dalle modalità di realizzazione, e ha lo scopo di rappresentare la realtà descritta dal problema, tramite un preciso linguaggio formale (ad esempio i diagrammi UML, ma anche il modello **Entity-Relationship (E-R)**)

Non esiste un modello corretto: la stessa realtà può essere modellata in modi diversi a seconda dell'interpretazione che si dà al problema. Tuttavia deve essere seguito uno specifico "percorso mentale" nella progettazione di una base di dati:

1. **Aspetto ontologico:** quanto conosco del dominio del discorso? Quanto sono familiare con l'ambito di discussione?
Durante questa fase, detta anche fase di *analisi del problema*, devo delineare quali sono le entità la cui realtà deve essere modellata e come si relazionano tra loro.

2. **Aspetto logico:** con quale concetto astratto voglio realizzare le entità del modello? (es. relazionale, gerarchico, reticolare, ...)
Durante questa fase definisco le effettive relazioni tra le entità.
3. **Aspetto linguistico:** con quale linguaggio formale definiamo il modello? (es. SQL/algebra relazionale)
4. **Aspetto pragmatico:** come memorizzo i dati nei supporti di memorizzazione fisica?

3.2 Entità, collezioni e attributi

Nella descrizione delle fasi da seguire durante la progettazione di una base di dati abbiamo citato un termine importante e fondamentale nella modellazione concettuale della realtà di riferimento: **entità**.

Definizione 3.2.1: Entità

Le *entità* di un problema sono quegli oggetti a cui il problema dà particolare rilevanza, e dei quali ci interessa rappresentare alcuni fatti a loro associati, detti per l'appunto **attributi (o proprietà) dell'entità**.

Ogni entità appartiene ad un **tipo** che ne specifica la natura. Ad esempio l'entità **Studente** appartiene alla categoria **Persona** (possiamo vederla come l'ereditarietà nella programmazione ad oggetti).

Una **collezione** di entità è invece un insieme variabile nel tempo di entità tutte dello stesso tipo. Ad esempio, se pensiamo a tutti gli studenti di un'università, quella rappresenta una collezione di entità di tipo **Studente**.

Supponiamo di dover creare la base di dati di un sistema di memorizzazione degli studenti, professori ed esami. Possiamo individuare nello studente un'**entità**, in quanto si tratta di uno degli oggetti fondamentali descritti dal problema, e nella sua matricola, cognome e data di nascita gli **attributi** fondamentali che lo descrivono.

Un entità o un attributo non mantengono il loro status costantemente all'interno di ogni analisi e di ogni problema: in una realtà potrebbe essere messa più importanza su un oggetto, spingendoci dunque a farlo diventare entità, mentre in un'altra tale importanza potrebbe mancare, rilegando così l'oggetto ad un semplice attributo. Ad esempio, proviamo a rappresentare una biblioteca: se ci interessasse modellare solo i libri, senza curarci degli autori, potremmo avere uno schema logico di questo tipo **Libro(titolo, annoDiPubblicazione, casaEditrice, autore)**. Se invece fossimo interessati anche agli autori, renderemmo quell'oggetto una vera e propria entità con nome e cognome, anziché lasciarlo un semplice attributo.

3.3 Le associazioni

Definizione 3.3.1: Associazione

Un'istanza di associazione è un fatto che collega due o più entità, stabilendo un legame logico fra loro.

Matematicamente, definiamo $R(X, Y)$ un'associazione fra due collezioni di entità X ed Y un insieme di istanze di associazione tra elementi $x \in X$ e $y \in Y$. Il prodotto cartesiano $X \times Y$ è detto **dominio dell'associazione**.

Generalmente, nella modellazione concettuale, le associazioni sono descritte da un *verbo* che lega le varie entità coinvolte. Chiaramente tale verbo deve essere esplicativo della natura dell'associazione, e per tale ragione si devono evitare verbi come "essere" o "avere" oppure verbi modali.

Un esempio di associazione possiamo vederlo anche con il precedente esempio della biblioteca: le entità **Libro** e **Autore** sono collegate tra loro dall'associazione **SCRIVERE**, in quanto è l'autore a scrivere il libro, e il libro viene scritto dall'autore.

Un'associazione è caratterizzata dalle seguenti proprietà strutturali:

- **Molteplicità:** prese le due collezioni di entità X ed Y e un'associazione $R(X, Y)$, indica quante istanze di un elemento $x \in X$ sono associate ad uno e un solo elemento $y \in Y$.

Formalmente, tale proprietà è descritta dal **vincolo di univocità**:

- Un'associazione si dice **univoca** rispetto ad X se per ogni elemento $x \in X$ esiste **al più** un elemento $y \in Y$ che è associato ad x .
- Se tale vincolo fallisce (ovvero se esiste **almeno** un elemento $x \in X$ a cui è associato più di un elemento $y \in Y$) l'associazione si dice **multivalore**.

Facendo un esempio, un'associazione del tipo $R(X, Y) = (1 : N)$ è multivalore su X e univoca su Y . Viceversa, se avessimo scritto $R(X, Y) = (N : 1)$, sarebbe stata univoca su X e multivalore su Y . Possiamo leggerla "a croce", diciamo: la molteplicità dell'elemento più a sinistra è il valore più a destra, mentre quella dell'elemento più a destra è il valore più a sinistra. Come nelle proporzioni, gli esterni vanno con gli esterni e gli interni con gli interni.

- **Totalità:** indica se l'associazione tra un elemento $x \in X$ e un elemento $y \in Y$ è opzionale oppure no. Formalmente:
 - Un'associazione $R(X, Y)$ si dice **totale** se, per ogni elemento $x \in X$ esiste *almeno* un elemento $y \in Y$ associato ad x (questa proprietà è conosciuta meglio come **obbligatorietà del valore**).

- Tale associazione si dice invece **parziale** quando il vincolo imposto precedentemente fallisce, ovvero quando ad un valore $x \in X$ *non è detto* che vi sia associato un valore $y \in Y$ (questa proprietà è più conosciuta come **opzionalità**).

Ad esempio, pensiamo ad un'associazione del tipo **Possedere(Persona, AutomobileConTarga)**. Riprendendo il discorso sulla molteplicità, questa sarà del tipo (1:N), in quanto una persona può possedere più di un'automobile con targa, però una singola automobile con targa è posseduta da una sola persona. Vedendo invece il vincolo di totalità, quest'associazione risulta essere *opzionale* sulla Persona, in quanto non è detto che una persona possieda un'automobile, mentre è *totale* su automobile, in quanto un'automobile già targata è sicuramente proprietà di qualcuno.

Vediamo adesso una delle numerose rappresentazioni grafiche con cui possiamo rappresentare le associazioni, e delle particolarità che le contraddistinguono:

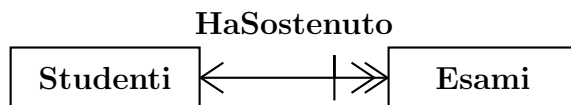


Figura 3.1: Relazione **Ha-Sostenuto**

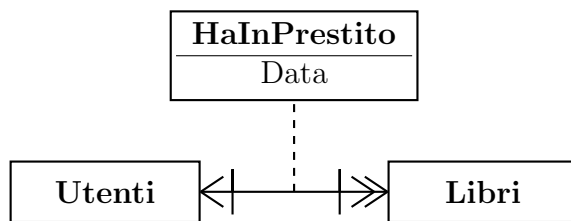


Figura 3.2: Relazione con attributi. L'attributo *Data* è agganciato alla relazione stessa.

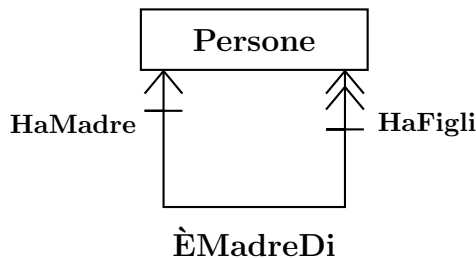


Figura 3.3: Relazione ricorsiva. L'entità **Persone** si relaziona con sé stessa tramite i ruoli *HaMadre* e *HaFigli*.

Descriviamo adesso tutte le informazioni ricavabili dalla notazione presa in esempio:

- Il singolo > oppure < significa che la relazione è **univoca** verso quel lato. Vedendo la figura 3.1 vediamo come in questa specifica modellazione un esame venga sostenuto da uno e un singolo studente.
- Il doppio >> oppure << significa che da quel lato la relazione è **multivalore**. Sempre prendendo in esempio gli studenti, uno studente può aver sostenuto più volte lo stesso esame. Anche con **Utenti - Libri**, vediamo come un utente prende in prestito più libri, mentre un libro viene preso in prestito solo da un utente alla volta.
- La barra verticale | vicino alla molteplicità indica che da quel lato la relazione è **parziale**, dunque opzionale. Ad esempio vediamo come, nel caso **Studenti - Esami** il singolo studente potrebbe ancora non aver mai sostenuto quell'esame, mentre l'esame stesso, essendo registrato, è sicuramente stato sostenuto da qualche studente.

3.4 Le gerarchie di classi

Spesso le classi di entità sono organizzate in una gerarchia di *specializzazione/generalizzazione*. Una classe della gerarchia minore di altre viene detta **sotto-classe**, mentre le altre prendono il nome di **superclassi**. Il concetto è lo stesso alla base dell'ereditarietà nella programmazione ad oggetti. Per fare un esempio pertinente, la classe degli *studenti universitari* può essere una generalizzazione (o sopraclasse) delle seguenti classi:

- Matricole e laureandi.
- Studenti in corso e studenti fuori corso.
- Studenti locali e studenti fuori sede.
- Studenti maschi e studentesse.

Fra i tipi oggetto è definita una **relazione di sottotipo** che regola la struttura gerarchica dei dati. Per il lettore questa è semplicemente una formalizzazione matematica del concetto di *polimorfismo* all'interno della programmazione ad oggetti.

Definizione 3.4.1: Relazione di sottotipo

Fra i tipi oggetto viene stabilita una relazione di sottotipo che soddisfa le seguenti proprietà fondamentali:

- È **asimmetrica**, **riflessiva** e **transitiva** (definendo così una relazione di ordine parziale).
- **Principio di sostitutività**: Se T è sottotipo di T' , allora gli elementi

appartenenti al tipo T possono essere usati in qualsiasi contesto in cui possano apparire valori di tipo T' .

In particolare, ciò implica che:

- Gli elementi di T possiedono tutte le proprietà degli elementi di T' .
- Per ogni proprietà p in T' , il suo tipo associato in T deve essere un sottotipo del suo tipo in T' .

Facendo nuovamente riferimento alla programmazione ad oggetti, anche nella modellazione concettuale possiamo definire degli oggetti (le entità del problema che vogliamo modellare) a partire da altri già esistenti. Questi nuovi oggetti ereditano dal proprio genitore (o *dai propri genitori*, in quanto viene supportata l'ereditarietà multipla) tutti i suoi attributi, e potranno anche definirne di nuovi o ridefinire gli esistenti.

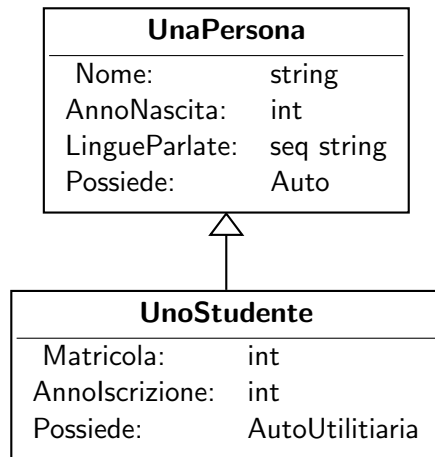


Figura 3.4: Ereditarietà tra classi. La classe **UnoStudiante** eredita da **UnaPersona**, specializzando il tipo dell'attributo *Possiede*.

Fra le classi può essere definita una relazione di **sottoclasse**, detta anche di **sottoinsieme**, caratterizzata dalle seguenti proprietà:

Definizione 3.4.2: Relazione di sottoclasse

La **relazione di sottoclasse** è definita dalle seguenti proprietà:

- È **antisimmetrica**, **riflessiva** e **transitiva**.
- Se C è sottoclasse di C' , allora il tipo degli elementi di C è sottotipo del tipo degli elementi di C' (**vincolo intensionale**).
- Se C è sottoclasse di C' , allora gli elementi di C sono un sottoinsieme degli elementi di C' (**vincolo estensionale**).

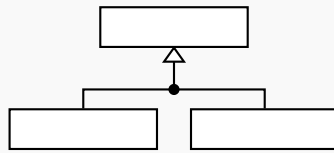
3.4.1 Vincoli su insiemi di sottoclassi

Per gli insiemi delle sottoclassi (i domini dei valori che tali classi possono assumere) vengono anche stabiliti dei vincoli.

Definizione 3.4.3: Vincolo di disgiunzione

Gli elementi di una sottoclasse non possono far parte di un'altra sottoclasse. Sostanzialmente stiamo dicendo che, avendo delle classi che sono specializzazioni di una sopraclasse, tali sottoclassi **non avranno nulla in comune**, e dunque un elemento di una sottoclasse non avrà nulla a che vedere con un elemento di un'altra sottoclasse.

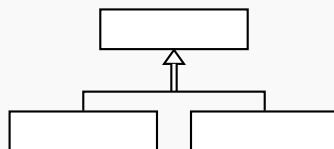
Pensiamo ad esempio di avere una classe **Persona** che ha come sottoclassi **Maggiorenni** e **Minorenni**. Tra queste sottoclassi vige il vincolo di disgiunzione, in quanto una persona non potrà **mai** essere contemporaneamente sia minorenni che maggiorenne. D'altro canto, se avessimo una classe **Persona** con sottoclassi **Programmatore** e **Maggiorenne**, tra queste due sottoclassi non vige il vincolo di disgiunzione in quanto non possiamo categoricamente dire che nessun programmatore è maggiorenne o viceversa. Formalizzando in maniera matematica questo vincolo, possiamo dire che *i domini delle due sottoclassi sono due insiemi la cui intersezione è nulla*.



Definizione 3.4.4: Vincolo di copertura

L'unione degli elementi delle sottoclassi contiene **tutti** gli elementi della sopraclasse.

Mantenendoci sempre sull'esempio fatto in precedenza con **Persona**, **Maggiorenne**, **Minorenni**, tra le due sottoclassi è presente un vincolo di copertura, in quanto una persona **o** è maggiorenne **o** è minorenni. Necessariamente una delle due. In modo più matematico e formale, possiamo dire che *l'unione dei domini delle sottoclassi è uguale al dominio della sopraclasse*.



Se tra le sottoclassi di una gerarchia si verificano contemporaneamente il vincolo di disgiunzione e quello di copertura, si dice che l'insieme delle sottoclassi è una **partizione** della classe padre.